جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 1- Introduction

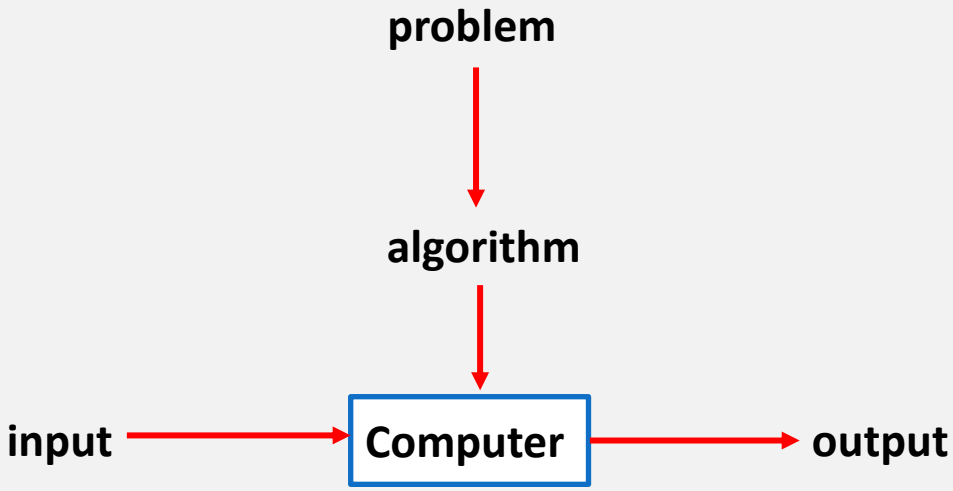
References:

- *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*
- *The Algorithm Design Manual*, Steven S. Skiena. Springer 2nd Edition.
- Introduction to Algorithms, Thomas H. Cormen, MIT press 3rd Edition

Introduction to Algorithms Design

What is an algorithm?

- A step-by-step procedure to solve a problem
- Any well-defined computational procedure that takes some value, or set of values, as ***input*** and produces some value, or set of values, as ***output***.
- The idea behind any reasonable computer program.



Introduction to Algorithms Design

Special Features of Algorithms

- Non-ambiguity
- Range of inputs
- Same algorithm can be represented in different ways
- Several algorithms for solving the same problem
- Different algorithms with different speeds

Introduction to Algorithms Design

The greatest common divisor

Problem: Find gcd(*m,n*), the greatest common divisor of two nonnegative, not both zero integers *m* and *n*

Examples: gcd(60,24) = 12, gcd(60,0) = 60, gcd(0,0) = ?

- Many ways for computing gcd(m,n):
  1. Euclid's algorithm
  2. Consecutive integer checking algorithm
  3. Middle-school procedure

Introduction to Algorithms Design

The greatest common divisor

1. Euclid's algorithm

Euclid's algorithm is based on repeated application of equality

$$\gcd(m,n) = \gcd(n, m \bmod n)$$

until the second number becomes 0, which makes the problem trivial.

Example:

gcd(60,24) = gcd(24,60 mod 24)

= gcd(24,12) = gcd(12,24 mod 12) =

= gcd (12,0) = 12

- **Step 1** If $n = 0$, return m and stop; otherwise go to Step 2
- **Step 2** Divide m by n and assign the value fo the remainder to r
- **Step 3** Assign the value of n to m and the value of r to n. Go to Step 1.

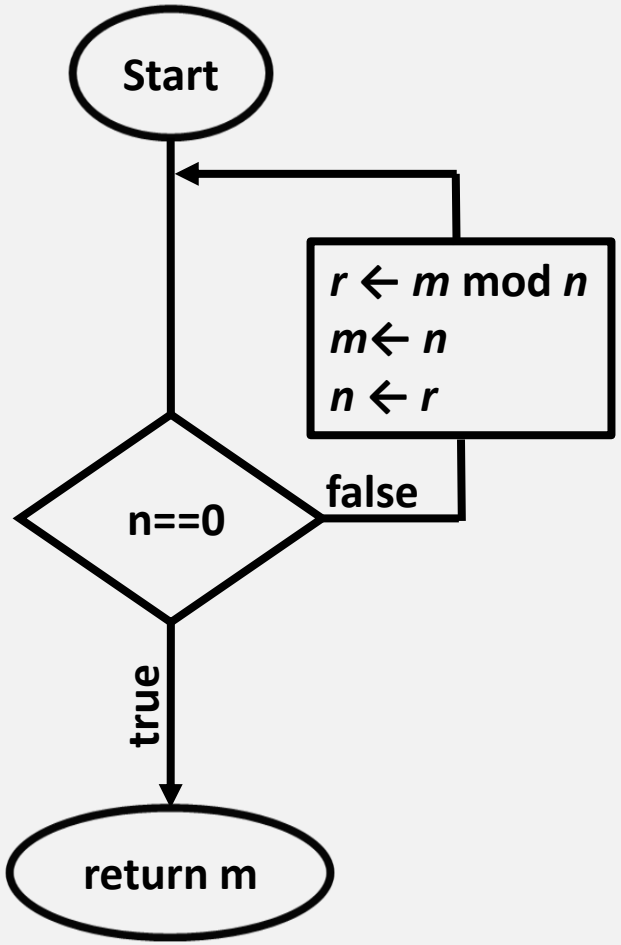
Introduction to Algorithms Design

The greatest common divisor

Euclid's algorithm implementation

recursive

```
int GCD(int m, int n) {
    if(n==0)
        return m
    return GCD(n, m mod n)
}
```



iterative

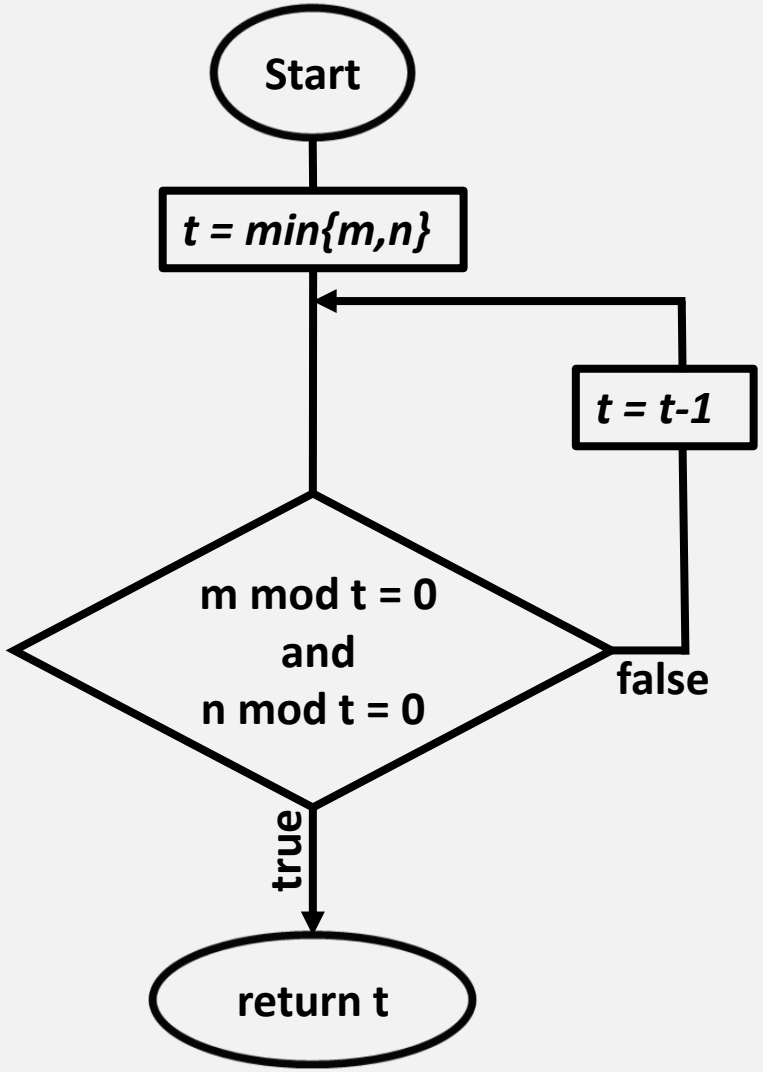
```
int GCD(int m, int n) {
    while n ≠ 0 do
        r ← m mod n
        m← n
        n ← r
    End while
    return m
}
```

Introduction to Algorithms Design

The greatest common divisor

2. Consecutive integer checking algorithm

- **Step 1** Assign the value of min{*m*,*n*} to *t* (A common divisor cannot be greater than the smaller of input numbers)
- **Step 2** Divide *m* by *t*. If the remainder is 0, go to Step 3; otherwise, go to Step 4
- **Step 3** Divide *n* by *t*. If the remainder is 0, return *t* and stop; otherwise, go to Step 4
- **Step 4** Decrease *t* by 1 and go to Step 2



⚠ **Does not work correctly when one of its input numbers is zero**

Introduction to Algorithms Design

The greatest common divisor

3. Middle School Procedure

- **Step 1** Find the prime factorization of *m*
- **Step 2** Find the prime factorization of *n*
- **Step 3** Find all the common prime factors
- **Step 4** Compute the product of all the common prime factors and return it.

Example: for the numbers 60 and 24, we get

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\gcd(60, 24) = 2 \cdot 2 \cdot 3 = 12.$$

- Much more complex and slower than Euclid's algorithm
- The prime factorization steps are not defined unambiguously
- Step 3 is also not defined clearly enough. How would you find common elements in two sorted lists?

Introduction to Algorithms Design

The greatest common divisor

Middle School Procedure

Why the middle-school GCD method is inefficient and motivate better algorithms?

- **Step 1** Find the prime factorization of *m*
- **Step 2** Find the prime factorization of *n*

?

Check if x is a prime number

```
boolean isPrime(int x) {
    for( int i =2; i<x; i++){
        if( x mod i == 0)
            return false
    }
    return true
}
```

List of prime numbers less than N

```
void primeList(int N) {
    for( int i =2; i<N; i++){
        if( isPrime(i))
            print(i)
    }
}
```

Time Complexity

isPrime(x) → **O(x)**

primeList(N) → **O(N^2)**

Prime factorization of m and n → **Very slow for large numbers**

Introduction to Algorithms Design

The greatest common divisor

جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

Middle School Procedure

- **Step 1** Find the prime factorization of *m*
- **Step 2** Find the prime factorization of *n*

Sieve of Eratosthenes Algorithm

Input: Integer $n \geq 2$

Output: List of primes less than or equal to *n*

for $p \leftarrow 2$ to *n* do $A[p] \leftarrow p$

for $p \leftarrow 2$ to $\lfloor \sqrt{n} \rfloor$ do

    if $A[p] \neq 0$ //*p* hasn't been previously eliminated from the list

        $j \leftarrow p * p$

        while $j \leq n$ do

            $A[j] \leftarrow 0$ //mark element as eliminated

            $j \leftarrow j + p$

Introduction to Algorithms Design

The greatest common divisor

Middle School Procedure

- **Step 1** Find the prime factorization of *m*
- **Step 2** Find the prime factorization of *n*

Sieve of Eratosthenes Algorithm

Example: the list of primes not exceeding n = 25:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	**3**		5		7				11		13				17		19				23		25
2	3		**5**		7				11		13				17		19				23		

If one or both input numbers are equal to 1, the gcm algorithm will not work

Introduction to Algorithms Design

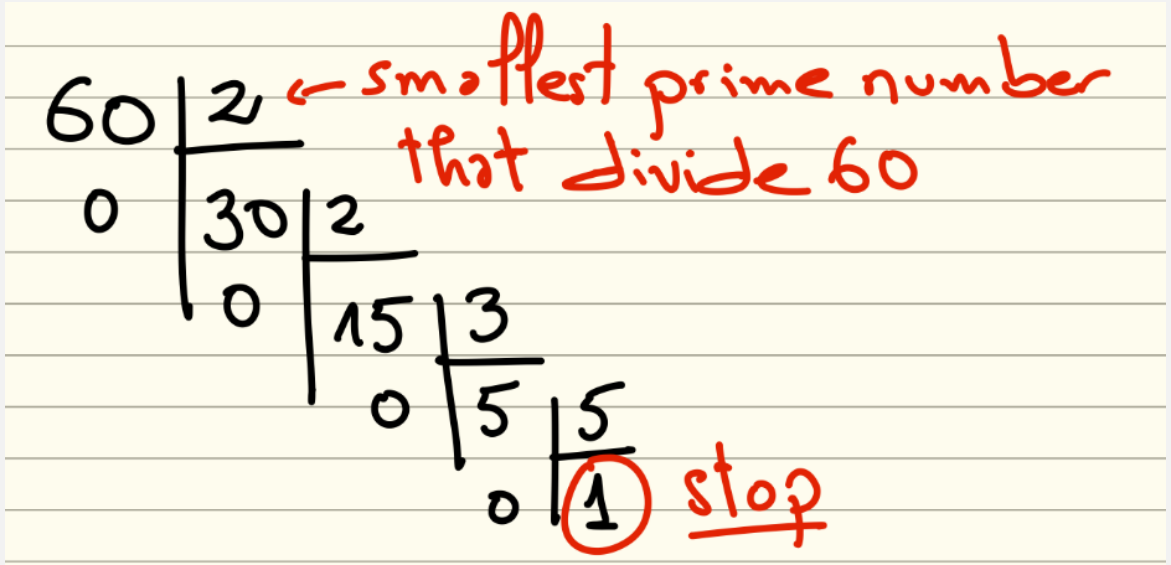
The greatest common divisor

Middle School Procedure

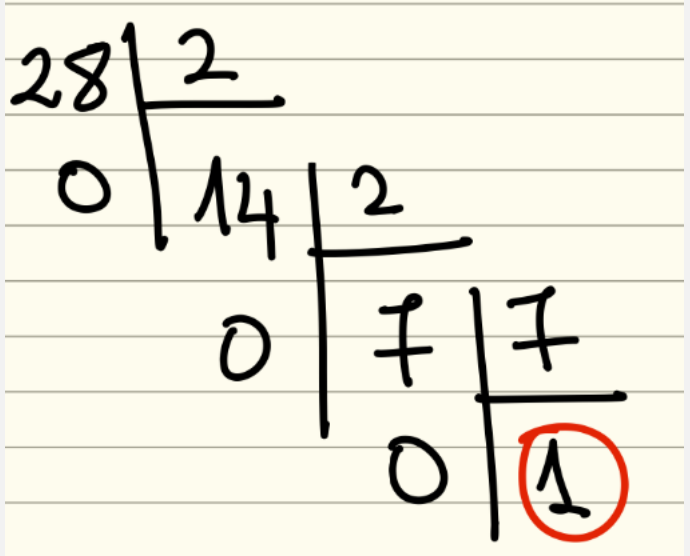
- **Step 3** Find the prime factors common to both numbers.

List of prime numbers									
2	3	5	7	11	13	17	19	23	29

60 = 2 . 2 . 3 . 5 ?



28 = 2 . 2 . 7 ?



GCD(60, 28) = 4

Introduction to Algorithms Design

The greatest common divisor

Middle School Procedure

Prime Factorization Using Prime Divisors

```
Input: integer n
Output: array factors[] containing prime factors of n

factors ← empty array
divisor ← 2

while n > 1 do
    if isPrime(divisor) then
        while n mod divisor = 0 do
            append divisor to factors
            n ← n / divisor
        end while
    end if
    divisor ← divisor + 1
end while

return factors
```

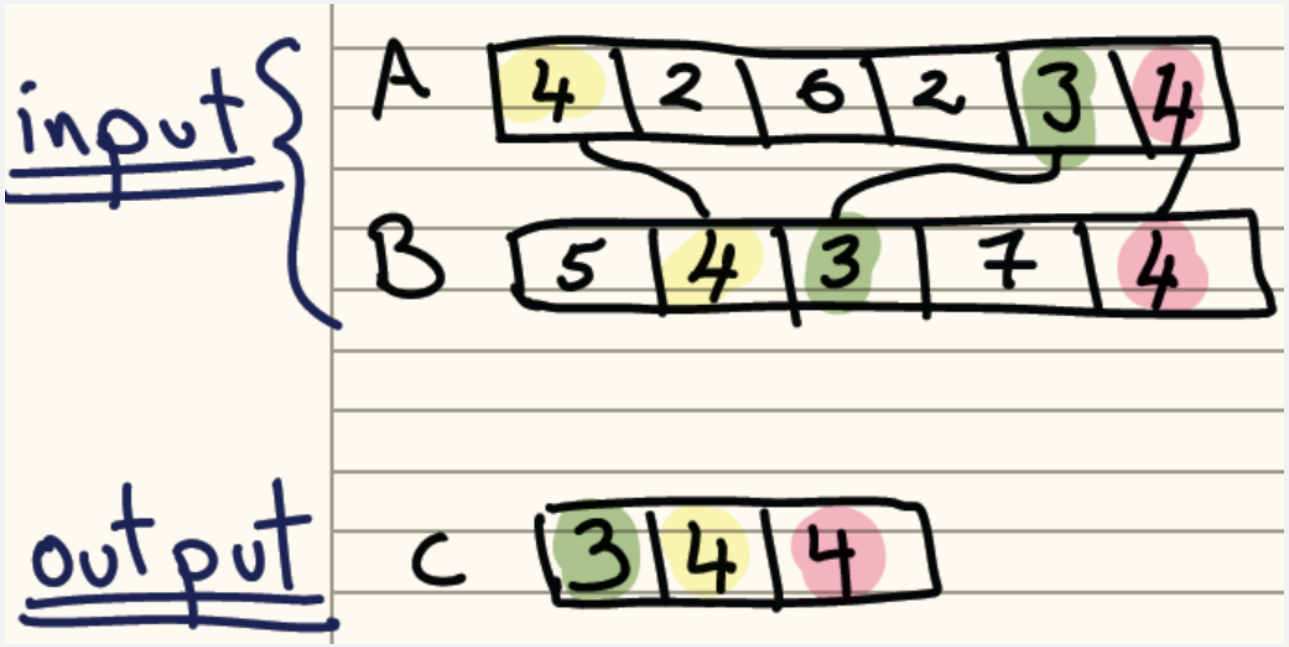
Introduction to Algorithms Design

The greatest common divisor

Middle School Procedure

- **Step 3** Find all the common prime factors

- **Input:** A, B (arrays of prime factors)
- **Output:** C (common prime factors)



Introduction to Algorithms Design

The greatest common divisor — Middle School Procedure

Common prime factors are selected **once per match**, not just by value but by **count**.

Array Comparison

```
Input: arrays A, B
Output: array C (common prime factors)

C ← empty array

for each element x in A do
    if x exists in B then
        append x to C
        remove x from B   // prevent reusing the same factor
    end if
end for

return C
```

Introduction to Algorithms Design

Why Algorithm Design is Challenging?

- Hard to design algorithms that are
  - Correct: always produces the right answer
  - Efficient: runs fast and uses reasonable resources
  - Implementable: can actually be coded and maintained

- To handle these challenges, we need to understand:
  - Design and modeling techniques
  - Resources - don't reinvent the wheel

*Algorithm design is not just about writing code — it is about making **smart choices and tradoffs**.*

Introduction to Algorithms Design

Algorithm Correctness

- How do you know an algorithm is correct?
  - produces the correct output for **every possible valid input**
- Since there are usually infinitely many inputs, it is not trivial
- Saying "it's obvious" can be dangerous
  - often one's intuition is tricked by one particular kind of input

*Therefore, we need **formal reasoning or proofs** to establish correctness.*

Introduction to Algorithms Design

Correctness: Tour Finding Problem

- Given a set of n points in the plane, what is the shortest tour that visits each point and returns to the beginning?
  - application: robot arm that solders contact points on a circuit board; want to minimize movements of the robot arm
- How can you find it?

Introduction to Algorithms Design

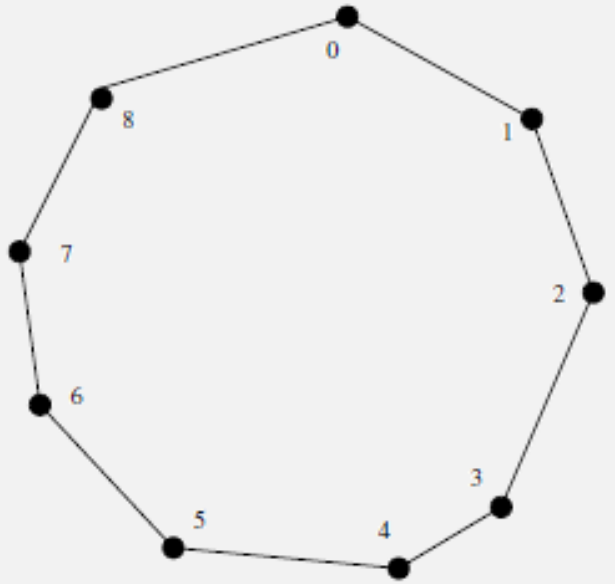
Correctness: Tour Finding Problem - Nearest Neighbor

Nearest Neighbor Algorithm

1.Start at any city

2.Repeatedly visit the nearest unvisited city

3.Return to the start



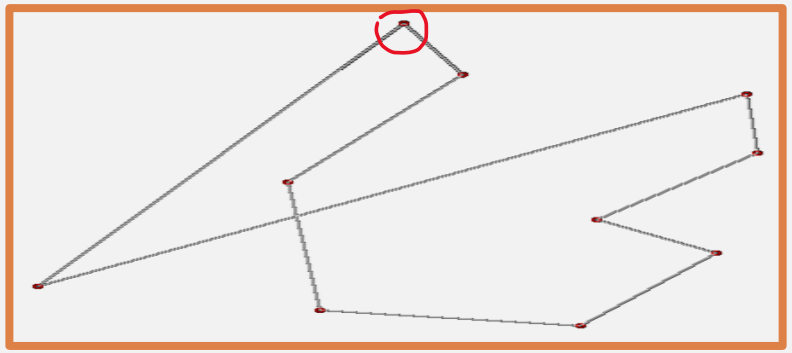
Introduction to Algorithms Design

Correctness: Tour Finding Problem - Nearest Neighbor

Input: A set of points (cities) in a plane

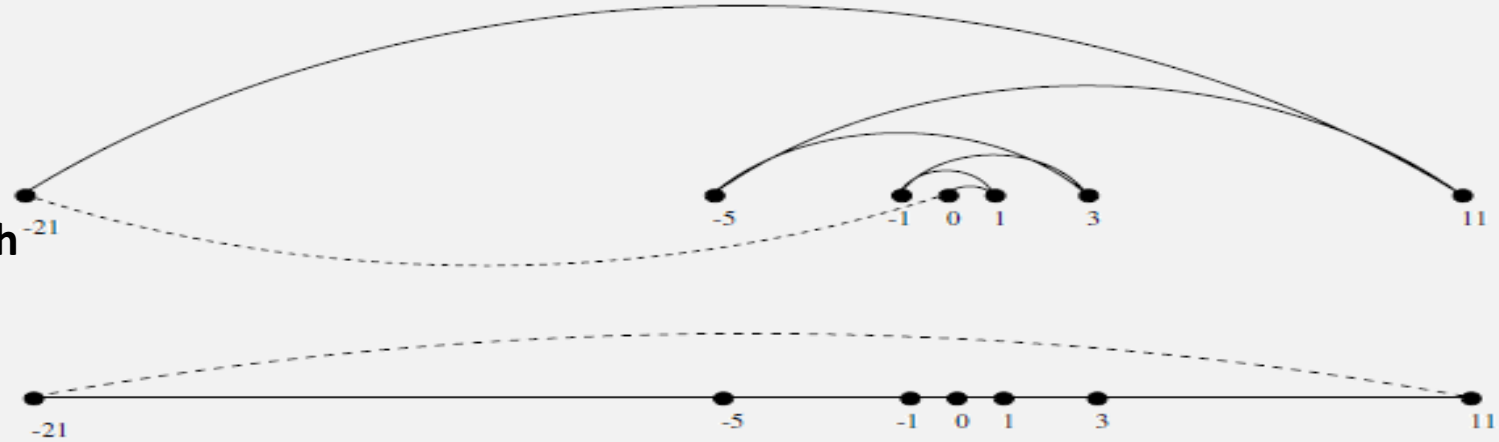
A tour constructed using the NN algorithm:

- Start point (Red circle)
- Go to the closest unvisited point.



- Points lie on a number line
- NN chooses the closest point each time
- **Result**:
  - The algorithm "zigzags"
  - Produces a **much longer path** than necessary

➡️ This is a **counter-example** where NN clearly fails.



⚠ **Seeking counter-examples to proposed algorithms is important part of design process**

Introduction to Algorithms Design

Algorithm Efficiency

- Algorithm Efficiency: describes how much **time** and **memory** an algorithm requires as the input size grows.
- Software is always outstripping hardware
  - need faster CPU, more memory for latest version of popular programs
- Given a problem:
  - what is an efficient algorithm?
  - what is the most efficient algorithm?
  - does there even exist an algorithm?

Even with faster hardware, poorly designed algorithms do not scale well as data size increases.

Introduction to Algorithms Design

How to Measure Efficiency

- Machine-independent way:
  - analyze "pseudocode" version of algorithm
  - assume idealized machine model
    - one instruction takes one time unit
  - Allows ***fair comparison*** between algorithms regardless of hardware

- "Big-Oh" notation:
  - describes how running time grows as input size increases, ignoring constants and lower-order terms.

- Worst-case analyses:
  - provides a guaranteed upper bound on performance
  - safe, often occurs most often, average case often just as bad

Introduction to Algorithms Design

Faster Algorithm vs. Faster CPU

- Algorithm S1 (slow algorithm, fast computer):

  Number of Keys = n

  Time Complexity = $2n^2$ instructions

  Computer Speed = 1 billion instructions/sec

  → **Very slow**, despite a powerful CPU

For $n = 1{,}000{,}000$:

$$2n^2 = 2 \times (10^6)^2 = 2 \times 10^{12} \text{ instructions}$$

Execution time:

$$\frac{2 \times 10^{12}}{10^9} = 2000 \text{ seconds}$$

- Algorithm S2 (fast algorithm, slower computer):

  Number of Keys = n

  Time Complexity = 50nlog2n instructions

  Computer Speed = 10 million instructions/sec

  → **20× faster**, even on a much slower machine.

For $n = 1{,}000{,}000$:

$$\log_2(10^6) \approx 20$$

$$50 \times 10^6 \times 20 = 10^9 \text{ instructions}$$

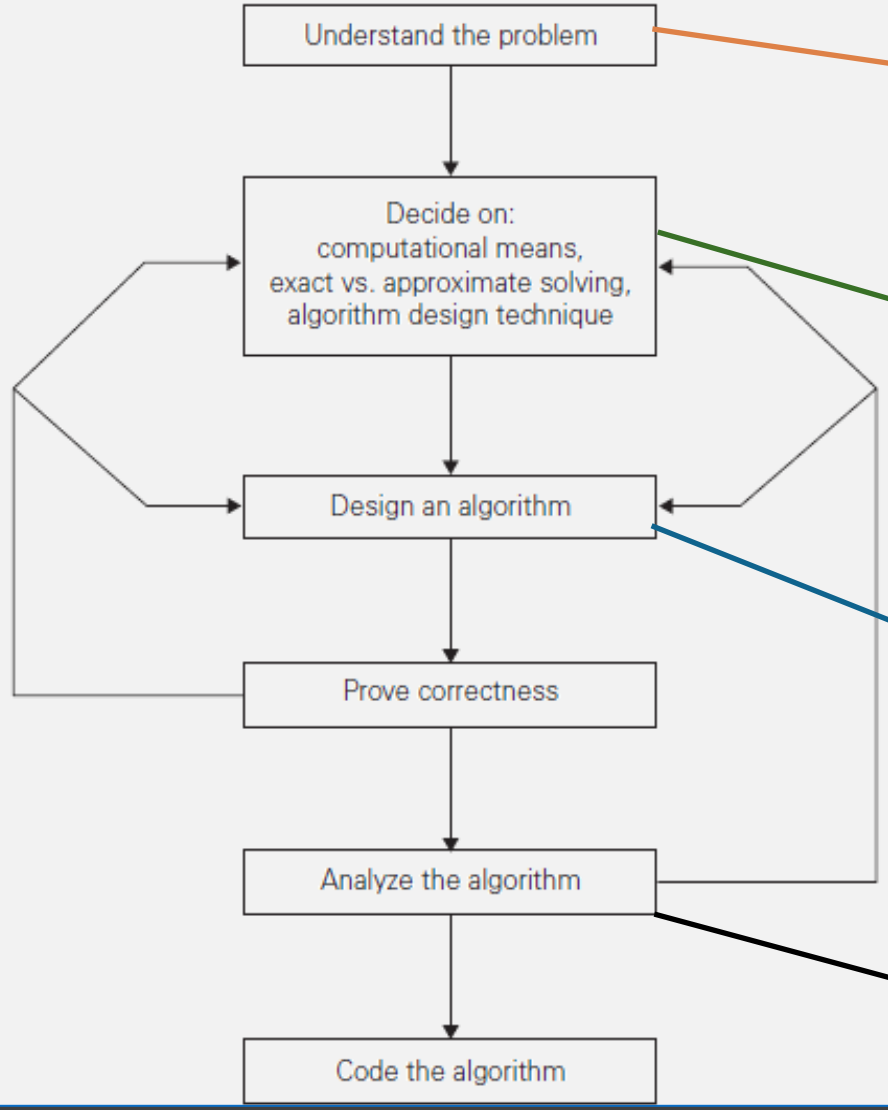
Execution time:

$$\frac{10^9}{10^7} = 100 \text{ seconds}$$

A faster algorithm running on slower hardware will outperform a slower algorithm on faster hardware for sufficiently large inputs.

Introduction to Algorithms Design

Algorithm design and analysis process



- Do few small examples by hand
- Think about special cases
- Specify the set of instances

- Capabilities of the computational device
  - Sequential Algorithm
  - Parallel Algorithm
- Choosing between exact and approximate algorithm
- Choosing the General approach to solve the problem

- Adapting a general technique
- Combine some techniques
- Choosing appropriate data structures

- Time efficiency
- Space efficiency
- Simplicity
- Generality

Introduction to Algorithms Design

Algorithm Design Techniques

- Brute Force & Exhaustive Search
  - follow definition / try all possibilities
- Divide & Conquer
  - break problem into distinct subproblems
- Transformation
  - convert problem to another one
- Dynamic Programming
  - break problem into overlapping subproblems
- Greedy
  - repeatedly do what is best now
- Iterative Improvement
  - repeatedly improve current solution
- Randomization
  - use random numbers

Introduction to Algorithms Design

Some Important Problem Types

- Sorting
  - a set of items
- Searching
  - among a set of items
- String processing
  - text, bit strings, gene sequences
- Graphs
  - model objects and their relationships
- Combinatorial
  - find desired permutation, combination or subset
- Geometric
  - graphics, imaging, robotics
- Numerical
  - continuous math: solving equations, evaluating functions